

Conclusion to SQLAlchemy Relationships

 [_ \(https://github.com/learn-co-curriculum/python-p3-conclusion-to-sqlalchemy-relationships\)](https://github.com/learn-co-curriculum/python-p3-conclusion-to-sqlalchemy-relationships)  [_ \(https://github.com/learn-co-curriculum/python-p3-conclusion-to-sqlalchemy-relationships/issues/new\)](https://github.com/learn-co-curriculum/python-p3-conclusion-to-sqlalchemy-relationships/issues/new)

Learning Goals

- Use SQLAlchemy to join tables with one-to-one, one-to-many, and many-to-many relationships.

Conclusion

In this Canvas module, we learned how to use SQLAlchemy to build relationships between data models and persist them in our larger database schemas. SQLAlchemy has several important classes, methods, and keywords to remember when you're building relationships in your own databases:

- The `ForeignKey` class is used in the `Column` constructor to create foreign key relationships.
- The `relationship()` method generates an attribute within a data model that maps to a record from a separate data model.
- The `backref` and `back_populates` keywords generate attributes in the *related* data model that map to records from the current data model.
- The `uselist` keyword defaults to `True`, but creates a one-to-one relationship when set to `False`.
- The `Table` class can be used to quickly create data models; it is especially useful for creating association tables.
- The `secondary` keyword allows us to build relationships through an association table using only the names of the data models on either side.

SQLAlchemy is a robust and sometimes complex library that gives you the power to create and relate any number of database tables for your Python applications. It can certainly get overwhelming if you attempt to memorize every class, method, and keyword, so remember to lean on its [robust online documentation](https://docs.sqlalchemy.org/en/14/orm/)  [_ \(https://docs.sqlalchemy.org/en/14/orm/\)](https://docs.sqlalchemy.org/en/14/orm/) and the lessons from this Canvas module as you start out. Soon enough, you'll be able to build entire databases without ever looking away from VSCode- but there's no rush!

Next up in the curriculum, you will get a chance to explore [every programmer's favorite topic](https://c.tenor.com/NdezMLQplh0AAAAC/not.gif)  [\(<https://c.tenor.com/NdezMLQplh0AAAAC/not.gif>\)](https://c.tenor.com/NdezMLQplh0AAAAC/not.gif) - *algorithms!*

Resources

- [Python 3.8.13 Documentation](https://docs.python.org/3/)  (<https://docs.python.org/3/>)
- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/)  (<https://docs.sqlalchemy.org/en/14/orm/>)
- [Alembic 1.8.1 Documentation](https://alembic.sqlalchemy.org/en/latest/)  (<https://alembic.sqlalchemy.org/en/latest/>)
- [Basic Relationship Patterns - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)  (https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)